

VARIABLES, CONSTANTS, AND DATA TYPES

CONTENTS OF THE CHAPTER

- *The concepts of constants and variables.*
- *Variable classification.*
- *Naming rules.*
- *Basic data types.*
- *The concept of operators and types of operators.*

Chapter 2 covers the most fundamental concepts in Java, including constants, variables, data types, and operators. Java also provides rules and conventions for naming conventions such as variable names, class names, method names, and constant names. Adhering to these naming rules helps create valid declarations and makes the source code easier to read and maintain. This chapter also discusses the use of strings, including how to declare and initialize character strings and how to use methods to work with strings. The final part of the chapter covers the use of five common operators in Java: arithmetic operators, assignment operators, comparison operators, logical operators, and conditional operators.

2.1. NAMING IN JAVA

In the programs, **place Names** (also known as **identifiers**) are used to identify different components in a program. These can be variables, functions, classes, objects, methods, etc. A **name** is a string of one or more characters and usually must begin with a letter and typically does not allow special characters.

2.1.1. Naming Rules

Each programming language typically has different naming **conventions** . Java also has naming rules that must **be** followed, specifically as follows:

- Do not use keywords **in** the names (refer to the appendix for

keyword research).

- **letters** , **numbers** , the dollar sign (\$) , and the underscore (_) are allowed when naming.
- Names **cannot** begin with a number.

Here are some examples of valid naming conventions:

- age
- \$salary
- _structure
- First
- NUMBER 1

Here are some examples of invalid naming conventions:

- 123abc
- Document
- -salary
- @test

It's important to note that, not just Java, most programming languages don't allow names containing spaces. In that case, "HelloWorld" is a valid identifier, but "Hello World" is not. Java is case-sensitive; therefore, "HelloWorld," "helloworld," "HELLOWORLD," and "hElloWorLD" are all distinct names.

2.1.2. Naming Conventions

Besides the naming conventions that every programmer **must** follow, naming **should also** adhere to the following guidelines:

Class Naming Convention:

- It should start with a capital letter.
- It is a noun (like Color, Button, System, Topic).
- Use proper words, instead of abbreviations.

For example:

```
class Employee{  
}
```

Method naming conventions:

- Start with a lowercase letter.
- It is a verb like main(), print(), println().
- If the name contains multiple words, start it with a lowercase letter followed by an uppercase letter, such as (camelCase) as in actionPerformed().

For example:

```
class Employee{  
    //method  
    Void draw(){  
    }  
}
```

Variable Naming Conventions:

- Start with a lowercase letter like id, name.
- If the name contains multiple words, start with a lowercase letter followed by an uppercase letter, such as firstName, lastName, etc.
- Avoid using single-character variables like x, y, z, or variables that don't clearly describe their meaning.

For example, declaring a variable as an attribute of the Employee class:

```
class Employee{  
    //variable  
    int id;  
}
```

Interface Naming Convention:

- It should start with a capital letter (usually the letter I).
- This is followed by an adjective such as Runnable, Remote,

ActionListener.

- Use proper words, instead of abbreviations.

For example:

```
interface IPrintable{  
}
```

Package Naming Convention:

- The name should be a lowercase letter like java, lang.
- If the name contains multiple words, they should be separated by dots (.), such as java.util, java.lang.

For example:

```
package com.javatpoint; //package  
class Employee{  
}
```

Conventions for naming Hang (Constant):

- It should be written in capital letters like RED, YELLOW.
- If the name contains multiple words, they should be separated by underscores (_), such as MAX_PRIORITY.

For example:

```
class Employee{  
    //constant  
    static final int MIN_AGE = 18;  
}
```

2.2. VARIABLES

2.2.1. The concept of variables

A variable is a value **that can change** within a program. A variable is allocated a **memory location** in main memory to store its value. Each variable must be given a unique name (identifier). To declare a variable, you need to specify its data type. For example:

```
int x = 15;
```

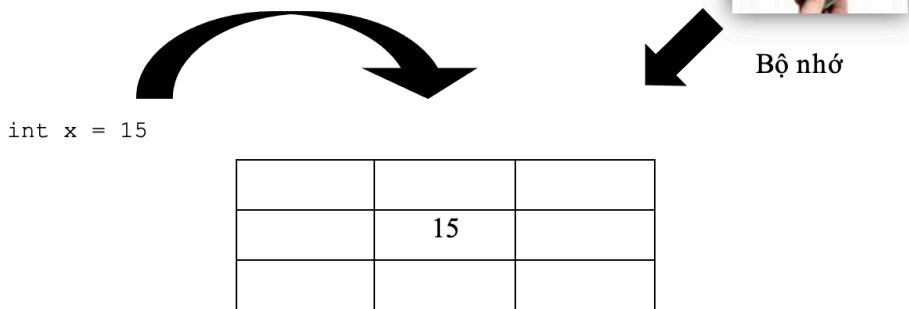


Figure 2 -1. Variable accessing a memory region in main memory

In Java, the only way to put data into a variable is by using **an assignment statement** . An assignment statement uses **the assignment operator** “=” to assign data to a previously declared variable. Variable declarations usually include specifying **the data type** . However, since Java 10, variable declarations can be done with the keyword **`var`** . In this case, the variable's type will correspond to the data type assigned to that variable, for example:

```
var x = 15;
var y = 21.2f;
```

In Java, there are **three types of variables** :

- Local variable
- Property variables
- Static variables

Examples of declaring and initializing valid variables in Java:

```
int a, b, c; // Declare three variables of type int: a, b, and c.
int a = 5, b = 7; // For example, create a variable
byte A = 11;      // Start creating a byte type variable named A.
double pi = 3.14159; // Declare and assign a value of PI.
```

```
char nam = 'b'; // The variable nam is initialized with the
value 'a'.
```

2.2.2 . Local Variables

Local variables are variables declared within methods or code blocks. Local variables are created when these methods or code blocks are called and are automatically destroyed after the methods or code blocks have finished executing. Access modifiers cannot be used for local variables. Local variables are executed internally within the code block. For example, the local variable ``tuoi`` in the code below can only work within the ``tuoiCon()`` method:

```
public class Test{
public void tuoiCon(){
int age = 0;
age = age + 10;
System.out.println("Child's age is: " + age);
}

public static void main(String args[]){
Test test = new Test();
test.tuoiCon();
}
}
```

The result when running the program above:

Variables need to be initialized before use. No default values are assigned to local variables, so these variables should be declared and assigned an initial value before use. The following example uses the variable ``age`` before initializing it, so it will generate a compile-time error.

```
public class Test{
public void tuoiCon(){
int age;
age = age + 10;
```

```

System.out.println("Child's age is : " + age);
}

public static void main(String args[]){
Test test = new Test();
test.tuoiCon();
}
}

```

Compilation error message:

```

Test.java:4: variable number might not have been initialized
age = age + 10;
^
1 error

```

2.2.3. Attribute Variables

Property variables are declared within a class, but outside a method. For example, the variables `ten` and `hocphi` .

```

public class Student{
public String ten;
Private double tuition;

public Student (String nameSV){
ten = tenSV;
}

public void setHocPhi(double hp){
tuition fee = salary;
}

public void inThongTin(){
System.out.println("Name and address: " + name );
System.out.println("Course: " + course);
}
}

```

```
public static void main(String args[]){
Student sv1 = new Student("Nguyen Van Doan");
sv1.setHocPhi(4000);
sv1.inThongTin();
}
}
```

The result when running the program above:

Full name: Nguyen Van Doan

Tuition fee: 4000.0

property variables are created when an object is created using the **'new' keyword** and destroyed when the object is destroyed. These properties can be used by all methods, constructors, and code blocks within the class. However, the availability of these properties to other classes depends on the declaration of **access modifiers** .

Objects are allocated space in **the heap memory region** , and their properties are also allocated space regardless of whether they have values or not. When properties are not assigned values, they have **default values** . For numeric properties, the default value is 0; for boolean properties, the default value is false; and for the object itself, it is null. The values of these property variables can be assigned during declaration or in constructor methods.

Properties can be accessed directly **by** calling the variable name within that class. However, from other classes, it must be called using the full, valid name as follows: "Object Name.Property Name".

2.2.4. Static Variables

Static variables (variables) A **static** variable is declared with the **'static' keyword** within a class, but outside of methods, and is typically used as a class variable. Static variables are stored in **the 'static' domain**. **remember static** (It's part of the heap memory). Static variables are created when the program starts and destroyed when the program ends. This is extremely important because, unlike property variables, static variables **are independent of the object** . The default value of a static variable is the

same as that of a property variable. For numeric types, the default value is 0; for boolean types, it's false; and for objects, it's null. Values can be assigned during declaration or within methods. Additionally, values can be assigned within **static methods** . Static variables are rarely used, other than being declared as constants . Constants are variables that are declared as static variables and have the keyword **final** added . Constant variables never change from their initial value.

Static variables can be accessed by calling the class name: “Class name.Static variable name” or “Object name.Static variable name”, however the first method is more accurate.

When declaring static variables using the **final** keyword , those variables are constants and are usually written in uppercase. Note that Java has the **const** keyword , but it is not used to declare constants. The example below declares the constant **FACULTY**:

```
public class Student{
private static double tuition;
public static final String FACULTY = "Faculty";
public static void main(String args[]){
Tuition fee = 4000;
System.out.println(FACULTY + "average tuition fee: " + tuition
fee);
}
}
```

Running the program above gives us the following result:

The average tuition fee for the IT department is 4000.

In the example above, there are two static variables, where FACULTY is a constant. If the variables are accessed from an outer class, static variables should be accessed in the form: “Class Name.Static Variable Name”, for example: Student.FACULTY.

2.3. DATA TYPES

2.3.1. Basic Data Types

Basic data types are the simplest data types in Java. At any given time, a basic data type stores only a single value, without any other information. In Java, there are eight basic data types:

Table 2 -1. Basic Data Types in Java

Data type	Size (bits)	range of values	For example
over	8	-128 to 127	byte byteNumber = 100;
large	16	-32,768 to 32,767	short shortNumber = 10000;
int	32	-2,147,483,648 to 2,147,483,647	int intNumber = 100000;
dragon	64	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long longNumber = 100000l;
Balance	32	1.4E ⁻⁴⁵ to 3.4028235E ³⁸	float floatNumber = 9.08f;
double	64	4.9E ⁻³⁴ to 1.7976931348623157E ³⁰⁸	double doubleNumber = 9.08;
boolean	1	true/false (default is false)	boolean boolValue = true;
bra	16	0 to 65,535	char charValue = 'a';

Data types such as **byte** , **short** , **int** The **`long` variable** is used to store integers. The size and range of these variables are shown in the table above.

Boolean data type The **`char`** data type is used to store true or false values . The **`char`** data type, on the other hand, is used to store characters. Unlike C, the **`char`** type uses 16 bits, so it can represent 65,535 different characters.

Any numeric character containing a decimal point is a **double character** . The double type uses 8 bytes, equivalent to 64 bits. Meanwhile, the **float type** , used for floating-point data, is represented by 32 bits, with 1 sign bit, 8 exponent bits, and 23 bits for the numeric part.

To create a float , you must add "F" or "f" to the end of the number. For example, "1.2F" is an abbreviation for the value 1.2 in float.

```
float income;  
Income = 3223.56F;  
System.out.println(income);
```

When the program is run, the result is:

```
3223.56
```

2.3.2. The "Wrapper" data type

In Java, for each basic data type, there is a wrapper **data type** . **These are called wrappers because they "wrap" the basic data types** within a single object. Therefore, wrappers are data types that can both store single values and have additional methods. Below is a list of wrapper classes corresponding to each basic data type:

Table 2.2 -. References to the base data and corresponding Wrapper layers.

Basic data types	Corresponding Wrapper Layer
boolean	Boolean
bra	Character
over	Byte
large	Short
int	Integer
dragon	Long
Balance	Float
double	Double

For example, the number 8 can be stored using a basic data type, but it can also be wrapped using the Integer class. In that case, it will be provided with additional important methods. Furthermore, Wrapper classes also have useful methods for working with this data.

```
public static void main ( String [] args) {  
String str = "45" ;
```

```
Integer x = Integer. valueOf (str);  
    System . out .println( Integer . toHexString ( x ));  
}
```

2.3.3 . Input and Output of Data Types

In Java, the most common way to input data from the keyboard is to use a **Scanner object** . The first step is to create a Scanner object named 'print' to scan data from 'System.in' (the data stream from the keyboard).

```
Scanner in = new Scanner(System.in);
```

The next step will be to use the corresponding methods of this Scanner object to input data with different data types. For example:

- Use nextInt() to read an integer.
- Use nextDouble() to read a real number.
- Use next() to read a string.

```
int num1;  
double num2;  
String str;  
System. out .print( "Enter an integer: " );  
num1 = in.nextInt();  
System. out .print( "Enter a floating point: " );  
num2 = in.nextDouble();  
System. out .print( "Enter a string: " );  
str = in.next();
```

Similar to C and C++, Java also uses the " **printf** " method to output data types in a format, for example:

```
System. out .printf( "%s, Sum of %d & %.2f is %.2f \n " , str,  
num1, num2, num1+num2)
```

Execute the two code snippets above to input numbers and strings and print the output to the screen; the result will be as follows:

```
Enter an integer: 3
```

```
Enter a floating point: 2.3
```

```
Enter a string: Hello
```

```
Hello, Sum of 3 & 2.30 is 5.30
```

2.4 . CHARACTER STRINGS

A **string** is a continuous sequence of characters. In Java, the `String` class is used to create strings. Therefore, it can be seen that `String` is not a basic data type; it consists of many characters (`char`) and is understood as a derived data type, created from the basic character data type.

2.4.1. Initializing a character string

String object can be created by assigning it a string of characters enclosed in double quotes. For example:

```
String s1 = "Cat";  
String s2 = "Cat";
```

Java uses a `String Pool` (a special memory area within the `Heap`) to store strings in order to optimize storage. However, by default, Java does not store all string objects in the `String Pool`. This is the biggest difference between Java and Java. **Use assignment to create Strings** and use the word **`new` to create Strings** . Each time a string constant is created by assigning a value, the JVM first checks the `Pool` containing the string constant. If the string already exists in the `Pool`, a reference to the `Pool` is returned. If the string does not exist in the `Pool`, a new instance of the string is created and placed in the `Pool`.

For example, in the above case, **s1 will** create the string `"Cat"` in the `String Pool` because that string already exists in the `String Pool`, so the variable **s2** will reference this string. Meanwhile, if you create a string using the keyword ``new``, the created string will not be stored in the `String Pool`. For example:

```
String s3 = new String("Cat");
```

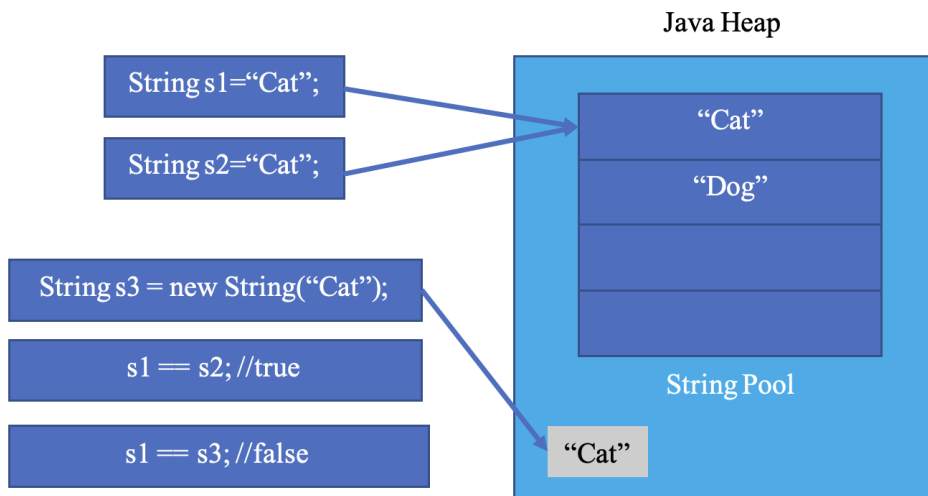


Figure 2.2 -. How String Pool works in Heap memory.

2.4.2. Methods for processing character strings

Java provides several methods for manipulating character strings. The following section will explore the functionality and usage of these methods.

The `format()` method Returns a String object. Use this method to create a formatted string. For example:

```
String fs;
float floatVar=4.5f;
int intVar = 9;
String stringVar="abc";
fs = String.format("The value of the float variable is " +
"%f, while the value of the integer variable " +
"The variable is %d, and the string is " +
"is %s", floatVar, intVar, stringVar);
System.out.println(fs);
```

The result when running the program:

```
The value of the float variable is 4.500000, while the value
of the integer variable is 9, and the string is abc.
```

The `length()` method returns the length of a string by counting the

characters in the string. Syntax:

```
int length = string_name.length();
```

For example:

```
public static void main(String[] args) {
    String chuoi;
    int doDai;
    Scanner scanner = new Scanner(System.in);

    System.out.println("Enter any string from the keyboard: ");
    string = scanner.nextLine();

    // calculate string length
    doDai = chuoi.length();

    System.out.println("String " + chuoi + " has length = " +
    doDai);
}
```

We get the following result when running the program:

The string "hello" has a length of 5.

concat() method To concatenate two strings. In fact, Java has two ways to concatenate two strings. The first way is to use the + operator, and the second way is to use **the concat() method**.

Syntax:

```
String string3 = string1.concat(String string2);
```

For example:

```
public static void main(String[] args) {
    String string1 = "Happy", string2 = "new year!";
    String string3 = string1.concat(string2);
    System.out.println(chuoi3);
}
```

The result when running the program:

Happy New Year!

The charAt() method returns the character in a string according to its index. A string is a collection of characters, and the first character in a string has an index of 0. For example, if a string has a length of 10, the indices of the characters in that string will be numbered from 0 to 9. The syntax of the charAt() method is:

```
char character = string.charAt(int index);
```

For example:

```
public static void main(String[] args) {  
    String string = "Happy new year!";  
    char character = chuoi.charAt(4); // Returns the character with  
    index 4 in the string  
    System.out.println(character);  
}
```

The result after running the program:

y

The compareTo() method compares two strings, string1 and string2, and returns the result.

- If result = 0, then the two strings are equal.
- If result < 0, then string1 < string2.
- If result > 0, then string1 > string2.

Syntax:

```
int result = string1.compareTo(String string2);
```

For example:

```
public static void main(String[] args) {  
    int result;  
    String string1 = "Happy new year!";  
    String string2 = "Happy new year!";  
  
    result = string1.compareTo(string2);  
    if (result == 0) {  
        System.out.println("String " + string1 + " = " + string2);  
    } else if (result < 0) {
```



```
System.out.println("String " + string1 + " < " + string2);  
else {  
System.out.println("String " + string1 + " > " + string2);  
}  
}
```

Result:

Happy new year series! = Happy new year!

The indexOf() method returns the position of the first occurrence of string2 in string1. If string2 is not present in string1, the result returned is -1.

Syntax:

```
int result = string1.indexOf(String string2);
```

For example:

```
public static void main(String[] args) {  
int result;  
String string1 = "Happy new year!";  
String string2 = "new year!";  
  
result = string1.indexOf(string2);  
System.out.println("First position of string " + string2 +  
" in string " + string1 + " = " + result);  
}
```

Result:

The first position where the string "new year!" appears in the string "Happy new year!" is 19.

The replace() method will replace the character you want to replace with another character in the string. If the character to be replaced is not present in string1, it will return string1.

Syntax:

```
string1.replace(char oldChar, char newChar);
```

For example:

```
public static void main(String[] args) {  
    String string1 = new String("Happy new year!");  
  
    // The replacement character 'l' is not present in string1  
    System.out.println(string1.replace('l', 'r'));  
  
    // Replace the character 'y' in string1 with 'r'  
    System.out.println("The string after replacement is " +  
        string1.replace('y', 'r'));  
}
```

Result:

```
Happy New Year!  
The replacement string is Happy new rear!
```

The trim() method removes leading and trailing spaces from string1. If the string is free of extra spaces, the program returns the original string.

Syntax:

```
String string1 = string1.trim();
```

For example:

```
public static void main(String[] args) {  
    String string1 = new String(" Welcome to Vietnam! ");  
    // Remove extra spaces from string1  
    string1 = string1.trim();  
    System.out.println("The string after removing extra spaces is "  
        + string1);  
}
```

Result:

```
The string after removing the extra spaces is Welcome to  
Vietnam!
```

The substring() method creates a substring starting at the position with index beginIndex and ending at the position with index endIndex in the parent string.

Syntax:

```
String stringChild = stringParent.substring(int beginIndex, int  
endIndex);  
// or from beginIndex to the end of the parent string  
String stringChild = stringParent.substring(int beginIndex);
```

For example:

```
public static void main(String[] args) {  
String string = new String("Welcome to NEU!");  
String stringCon1 = stringChain.substring(11);  
System.out.println(sequenceCon1);  
}
```

Result:

NEU!

Besides the methods above, Java has many other methods for working with strings.

2.5. OPERATORS

2.5.1. The concept of operators

Operators are used to perform operations on variables **and** values . In the example below, we use the '+' operator to add two values together:

```
public class Main {  
public static void main(String[] args) {  
int x = 100 + 50;  
System.out.println(x);  
}  
}
```

Although the '+' operator is commonly used to add two values together, as in the example above, it can also be used to add a variable and a value or a variable and another variable:

```
public class Main {
```

```

public static void main(String[] args) {
    int sum1 = 100 + 50;
    int sum2 = sum1 + 250;
    int sum3 = sum2 + sum2;
    System.out.println(sum1);
    System.out.println(sum2);
    System.out.println(sum3);
}
}

```

Result:

```

150
400
800

```

Java divides operators into the following groups:

- Arithmetic operators
- Assignment operator
- Comparison operators
- Logical operators
- Conditional operators
- Bitwise operators

2.5.2. Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations .

Table 2.3 -. List of arithmetic operators

Operat or	Describe	For example
+	Add two operands	x + y
But	Subtract two operands	x - y

*	Multiply two operands	$x * y$
/	Divide the two operands	x / y
%	Divide and take the remainder.	$x \% y$
++	Increase the value of the operand by 1.	$++x$
--	Decrease the value of the operand by 1 unit.	$--x$

2.5.3. Assignment Operator

Assignment operators are used to assign values to variables. In the example below, use the assignment operator (=) to assign the value 10 to a variable named x:

```
int x = 10;
```

Table 2 -4. List of assignment operators

Operator	For example	Equivalent
=	$x = 5$	$x = 5$
+=	$x += 3$	$x = x + 3$
-=	$x -= 3$	$x = x - 3$
*=	$x *= 3$	$x = x * 3$
/=	$x /= 3$	$x = x / 3$
%=	$x \% = 3$	$x = x \% 3$
&=	$x \& = 3$	$x = x \& 3$
=	$x = 3$	$x = x 3$
^=	$x = 3$	$x = x \wedge 3$
>>>	$x >> = 3$	$x = x >> 3$

2.5.4. Comparison Operators

Comparison operators are used to compare two values.

Table 2.5 -. List of comparison operators

Operator	Describe	For example
----------	----------	-------------

==	Comparison of equality	x == y
!=	Comparison of differences	x != y
news	Greater than comparison	x > y
<	Comparison of less than	x < y
>=	Comparison: greater than or equal to	x >= y
<=	Comparison: less than or equal to	x <= y

2.5.5. Logical Operators

Logical operators are used to define logic between variables or values.

Table 2 -6. List of logical operators

Operato r	Describe	For example
&&	Returns true if both operators are true.	x < 5 && x < 10
	Returns true if any of the operators are true.	x < 5 x < 4
!	Reverse the result	!(x < 5 && x < 10)

2.5.6. Conditional Expressions

Any good programming language has a few handy little features that make writing code easier. Java has the Ternary Operator , a **ternary operator** that makes writing if statements more concise. Its structure is as follows:

Expression? Command 1: Command 2

The computer checks the value of **the expression** . If the value is true, it executes **command 1** ; if false, it executes **command 2** .

For example:

next = (N % 2 == 0) ? (N/2) : (3*N+1);
--

The above statement will assign the value N/2 to next if N is even

(i.e., if $N \% 2 == 0$ is true) and it will assign the value $(3 * N + 1)$ to next if N is odd. (The parentheses in this example are not required, but they make the expression easier to read).

Consider another example:

```
public class Main {
public static void main(String[] args) {
int time = 20;
String leads;
result = (time < 18) ? "Good day." : "Good evening.";
System.out.println(result);
}
}
```

Results after running:

```
Good evening.
```

2.5.7. Bitwise Operators

Bitwise operators are used to work with the single bit value of a number. These operators can be used with integer data types such as int, short, char, long, etc.

Operator	Describe	For example
~	Reverse the value of the bit.	~ x
&	Returns 1 if there are 2 bits of 1.	x & y
^	Returns 1 if the two bits are different.	x ^ y
	Returns 1 if either of the two bits is 1.	x y
<<	Translate the left page bits	x << 2
>>	Shift the bits to the right.	x >> 2

For example:

```
int x = 20 ; //10100
```

```
int y = 4 ; //00100
System . out .println( "~x = " + ~ x );
System . out .println( "x & y = " + ( x & y ));
System . out .println( "x ^ y = " + ( x ^ y ));
System . out .println( "x | y = " + ( x | y ));
System . out .println( "x << y = " + ( x << y ));
System . out .println( "x >> y = " + ( x >> y ));
```

Result:

```
~x = -21
x & y = 4
x ^ y = 16
x | y = 20
x << y = 320
x >> y = 1
```

PRACTICE

Lesson 1. Identifying information about data types

This exercise will involve writing code to print the maximum, minimum, and bit length of basic data types. Begin by creating a TypesMinMax class with a main() method.

```
public class TypesMinMax {
    public static void main(String[] args) {
    }
}
```

In the main() method, print the Max and Min values of the int type:

```
System . out .println("int(min) = " + Integer . MIN_VALUE );
System . out .println("int(max) = " + Integer . MAX_VALUE );
System . out .println( "int(bit-length) = " + Integer . SIZE
);
```

Print the maximum and minimum values for the byte type:

```
System. out .println( "byte(min) = " + Byte. MIN_VALUE );
```



```
System.out.println( "byte(max) = " + Byte.MAX_VALUE );  
System.out.println( "byte(bit-length)=" + Byte.SIZE );
```

Print the maximum and minimum values for the short data type:

```
System.out.println( "short(min) = " + Short.MIN_VALUE );  
System.out.println( "short(max) = " + Short.MAX_VALUE );  
System.out.println( "short(bit-length) = " + Short.SIZE );
```

Print the Max and Min values of the long type:

```
System.out.println( "long(min) = " + Long.MIN_VALUE );  
System.out.println( "long(max) = " + Long.MAX_VALUE );  
System.out.println( "long(bit-length) = " + Long.SIZE );
```

Print the maximum and minimum values of a char type (16-bit character or 16-bit unsigned integer):

```
System.out.println("char(min) = " + (int)Character.MIN_VALUE);  
System.out.println("char(max) = " + (int)Character.MAX_VALUE);  
System.out.println("char(bit-length) = " + Character.SIZE);
```

Print the maximum and minimum values of a float type:

```
System.out.println( "float(min) = " + Float.MIN_VALUE );  
System.out.println( "float(max) = " + Float.MAX_VALUE );  
System.out.println( "float(bit-length) = " + Float.SIZE );
```

Print the maximum and minimum values for a double data type:

```
System.out.println( "float(min) = " + Float.MIN_VALUE );  
System.out.println( "float(max) = " + Float.MAX_VALUE );  
System.out.println( "float(bit-length) = " + Float.SIZE );
```

When you execute the program, the output will show information about the base data type:

```
int(min) = -2147483648  
int(max) = 2147483647  
int(bit-length) = 32  
byte(min) = -128  
byte(max) = 127
```

```
byte(bit-length)=8
short(min) = -32768
short(max) = 32767
short(bit-length) = 16
long(min) = -9223372036854775808
long(max) = 9223372036854775807
long(bit-length) = 64
char(min) = 0
char(max) = 65535
char(bit-length) = 16
float(min) = 1.4E-45
float(max) = 3.4028235E38
float(bit-length) = 32
```

Lesson 2. Comparing strings

Create a StringTest class with the main() method:

```
public class StringTest {
    public static void main(String[] args) {
    }
}
```

Perform string declarations and comparisons:

```
String s1 = "Cat" ;
String s2 = "Cat" ;
String s3= new String( "Cat" );
String s4 = "Dog" ;
String s5 = "CAT" ;
System.out.println ( s1==s2);
System.out.println ( s1==s3);
System.out.println(s1.equals(s2));
System.out.println(s1.equals(s3));
System.out.println(s1.equals(s5));
```

```
System.out.println(s1.equalsIgnoreCase(s5));
```

Check the printed output:

```
true
false
true
true
false
true
```

Lesson 3. Operators

Create the OpTest class. Declare and initialize the variables:

```
int age = 18;
Double weight = 71.23;
int height = 191;
boolean married = false;
boolean attached = false;
char gender = 'm';
```

Check the relational and logical operators:

```
System.out.println(!married && !attached && (gender == 'm'));
System.out.println(married && (gender == 'f'));
System.out.println((height >= 180) && (weight >= 65) && (weight
<= 80));
System.out.println((height >= 180) || (weight >= 90));
```

Implement the program and check the results:

```
true
false
true
true
```

QUIZ

What is the appropriate data type for the number 20.15?	
dragon	
large	
Integer	
Balance	

The assignment operator is used to:	
Assign a value to a variable	
Assign a statement to a method	
Assign a name to a value.	
Assign a value to an expression	

The byte data type has the following range:	
0 to 256	
From 0 to 128	
From 0 to 8	
From -128 to 127	

A variable in Java can contain:	
The \$ character	
The @ symbol	
The character -	
The % symbol	

The keyword/break statement is used to:	
Break a block of code	
Skip a loop	
Exiting a loop or switch block	
End a branching order	

Which one is NOT a Wrapper class?	
--	--

Integer	
Byte	
a	
Float	

Declare a constant using the keyword:	
static	
final	
const	
front	

How many bytes does a double variable use?	
2	
4	
8	
16	

The Scanner object is used for:	
Search for information from variables	
Scan for potential errors.	
Scan data from an input stream.	
Retrieving information from the array	

A package name may contain:	
The period.	
Exclamation mark!	
Curly brackets {}	
Percentage symbol (%)	

What is the result when the following code is executed ?	
<code>int a = - 10 ;</code>	

<pre>int b = a < 0 ? - a : a ; System.out.println("a = " + b);</pre>	
b = -10	
b = 10	
b = 0	
b = 1	

What is the output of the following code snippet ? <pre>for (int i = 0 ; i < 26 ; i ++) { char letter = (char) ('a' + i); System . out .println(letter); }</pre>	
Display 25 letters a	
Display the 26 letters a	
Display the alphabet	
A compilation error occurred.	

What is the output of the following code snippet ? <pre>short high = Short . MAX_VALUE ; short low = Short . MIN_VALUE ; System.out.println(high - low);</pre>	
32767	
0	
65534	
65535	

What is the output of the following code snippet ? <pre>String strObj = new String("NEU"); String str = "NEU" ;</pre>	
---	--

<code>System.out.println(strObj.equals(str));</code>	
Print: true	
print: false	
Compilation error	
print the word NEU	

What is the result of executing the following code? <pre>String strObj = new String("NEU"); String str = "NEU" ; System.out.println(str == strObj);</pre>	
Print: true	
print: false	
Compilation error	
print the word NEU	

PRACTICE EXERCISES

Lesson 1. Calculating Taxes

The sales tax on each product is 8.25%. Write a program that takes the initial **price of a product as input and prints the amount to be paid for the tax and the price after** the tax is applied.

Lesson 2. The Overtime Payment Problem

Write a program that prompts the user to input **the number of working hours and the hourly rate** . Then, calculate and print **the total amount payable** to that employee.

Lesson 3. Converting Centimeters to Inch

Knowing that 2.54 cm = 1 inch, write a program to convert a number from **cm to inches** . The user will input the length in **cm** , and the program will display the result after conversion to **inches** .

REFERENCES

- [1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).
- [2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.
- [3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.
- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.